SOFTWARE REQUIREMENTS SPECIFICATION

for

SNAKE GAME

Version 1.1
Updated implementation-aligned draft
7 November 2026

Project	Snake Game — Software Engineering Mini-Project
Prepared by	Charan, Vrunda, Ved and Tushar
Course	Software Engineering Mini-Project
Department	Computer Science and Engineering, PES University, Bengaluru
Technology	C++17 and SFML 2.6.2
Document status	Updated implementation-aligned draft; team and instructor approvals pending

Document control notice: This revision updates implementation details and verification status while retaining the original requirement identifiers and use-case model. Approval fields remain pending until the appropriate people provide actual approval.

Document Control

Revision History

Version	Date	Author(s)	Change summary	Approval
1.0	12 Oct 2026	Project team	Initial project SRS	Pending
1.1	7 Nov 2026	Project team	Aligned implementation details, controls, acceptance summary and RTM with available verification evidence.	Pending

Approvals

Role	Name	Signature / Email	Date
Project guide	Adithya	To be completed	To be completed
Team representative	Vrunda	To be completed	To be completed

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, non-functional and security requirements for the Snake Game mini-project. It provides a shared basis for implementation, review and testing.

1.2 Scope

Snake Game is an implemented single-player desktop game in C++ using SFML. The player controls a snake using keyboard input, collects food, increases the snake's length and score, and avoids collisions with the play-area boundary and its own body. After a fatal collision, the game displays the game-over state and final score and allows the player to restart. Online services and database functionality are outside the intended scope.

1.3 Intended Audience and Reading Suggestions

The intended audience includes the project team, testers, course instructor and evaluators. Section 2 describes the product; Section 3 specifies external interfaces; Sections 4 and 5 define functional, non-functional and security requirements; Section 6 summarizes quality attributes and acceptance; Section 7 presents the use-case model; and Section 8 maps requirements to tests.

1.4 References

- Software Engineering Mini-Project Deliverables Part-1 instructions provided for the course.
- SFML documentation for the version selected by the project team.
- C++ compiler and language documentation relevant to the team's selected build environment.

1.5 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
SFML	Simple and Fast Multimedia Library
UML	Unified Modeling Language
RTM	Requirements Traceability Matrix
FPS	Frames per second
TC	Test Case

2. Overall Description

2.1 Product Perspective

The game is a standalone desktop application. SFML is used for the game window, event handling and graphics. The core game operates locally and does not require a network connection or database.

2.2 Product Functions

• Start a round and display the snake, food, play area and score.
• Accept keyboard input to control the snake's direction.
• Move the snake and prevent an immediate 180-degree reversal.
• Place food, detect food collection, grow the snake and update the score.
• Detect boundary and self-collisions and enter the game-over state.
• Display the final score, allow restart and close the application safely.

2.3 User Classes and Characteristics

Player: The primary user, who uses keyboard controls and reads the game state and score. The game is intended to be usable without specialist technical knowledge.
Developer / Tester: Team members who build the application, inspect its modules and verify requirements using the separate Test Plan.

2.4 Operating Environment

The documented verification environment is Windows 11 Home Single Language, build 26300, 64-bit, on an HP Victus 15-fa1xxx with Intel Core i5-13420H, 16 GB RAM, NVIDIA RTX 4050 Laptop GPU and Intel UHD Graphics. The build used C++17, Microsoft C/C++ compiler 19.52.36815 for x64 (Preview), CMake 4.3.1 and SFML 2.6.2. The project uses CMake configuration and Google Test for automated tests.

2.5 Design and Implementation Constraints

The project is implemented in C++ using SFML. Keyboard input is the primary control method. The application is a local desktop game; online play, accounts and database integration are outside the intended scope.

2.6 Assumptions and Dependencies

A compatible C++ build environment, SFML installation and graphical desktop environment are assumed to be available. Successful build and execution depend on the compiler, SFML version and build instructions selected by the team.

2.7 Implemented Controls and Game Rules

Action	Implemented behaviour
Start	Enter or Space starts a round from Ready.
Move	Arrow keys change direction while Playing; immediate opposite-direction reversal is rejected.
Restart	R or r restarts after Game Over or win/completion.
Exit	Escape or closing the window exits the application.
Scoring	10 points are awarded per food item.
Initial / restart state	Three-segment snake at grid coordinate (5,5), facing Right; current score resets to 0 and best session score is retained.

3. External Interface Requirements

3.1 User Interfaces

The game interface displays the play area, snake, food and current score. When a fatal collision occurs, the interface displays a game-over message and final score. Controls and restart instructions are documented in the interface or README.

3.2 Hardware Interfaces

The game uses a standard keyboard for input and a graphical display for output. No dedicated hardware device is required.

3.3 Software Interfaces

The application is built using a compatible C++ compiler and SFML 2.6.2 in the documented verification environment. The README provides build and run instructions.

3.4 Communications Interfaces

Not applicable to the intended core game. The game operates locally without network communication.

4. System Features and Functional Requirements

Each functional requirement is uniquely identified and has an observable acceptance criterion. Requirements use 'shall' statements so that they can be verified.

ID	Functional requirement	Acceptance criteria
FR-01	The system shall start a new round when the player invokes the agreed start action.	The game enters the playing state.
FR-02	The system shall display a bounded play area.	The play-area boundary is visible.
FR-03	The system shall place the snake fully inside the play area at the start of a round.	Every initial snake segment is inside the boundary.
FR-04	The system shall accept the documented keyboard keys to change the snake's direction.	A valid direction key changes direction according to the control rules.
FR-05	The system shall move the snake in its current direction during gameplay.	The head advances and the body follows.
FR-06	The system shall reject an immediate 180-degree direction reversal.	An opposite-direction input does not reverse the snake directly.
FR-07	The system shall place food at a valid position inside the play area that is not occupied by the snake.	Food appears inside the play area without overlapping the snake.
FR-08	The system shall detect when the snake reaches food.	One food-consumption event is registered for the food item.
FR-09	The system shall increase the snake's length after food is consumed.	At least one segment is added after a food-consumption event.
FR-10	The system shall increase the score by the configured value when food is consumed.	The displayed score increases by the configured food value.
FR-11	The system shall generate replacement food after food is consumed.	A new food item appears at a valid unoccupied position.
FR-12	The system shall detect a collision between the snake's head and the play-area boundary.	A boundary collision triggers the game-over state.
FR-13	The system shall detect a collision between the snake's head and its own body.	A self-collision triggers the game-over state.
FR-14	The system shall stop snake movement after a fatal collision.	The snake no longer moves after the game-over state begins.
FR-15	The system shall display a game-over message and the final score after a fatal collision.	The game-over message and final score are visible.
FR-16	The system shall allow the player to restart after game over.	A new round starts without relaunching the application.
FR-17	The system shall reset the snake, direction and score when a new round starts.	The new round begins with the defined initial state and score.
FR-18	The system shall close safely when the player closes the game window.	The application exits without crashing.

5. Non-Functional Requirements

ID / category	Non-functional requirement	Acceptance criteria / measurement
NFR-01 — Performance	The game shall average at least 30 frames per second during a 60-second normal gameplay run on the agreed test computer.	Measured average is at least 30 FPS during the defined run.
NFR-02 — Usability	The game controls and restart action shall be described in the game interface or README.	A reviewer can identify the controls and restart procedure from the supplied instructions.
NFR-03 — Reliability	The game shall complete 10 consecutive start/restart cycles without crashing.	All 10 cycles complete without a crash.
NFR-04 — Maintainability	The implementation shall separate movement, food, collision, game control and rendering responsibilities into identifiable modules or classes where practical.	Code review can identify these responsibilities in the implementation.
NFR-05 — Compatibility	The project shall build and launch using the compiler and SFML version documented by the team.	A clean build and launch succeed in the documented environment.

ID / category	Non-functional requirement	Acceptance criteria / measurement
NFR-06 — Documentation	The README shall list the required tools, SFML dependency and build/run steps.	A teammate can follow the README to prepare the environment and attempt to build and launch the game.

5.1 Security

5.1.1 Security Objectives

Objective ID	Security objective
SO-01	Prevent unsupported keyboard input or invalid game-state changes from causing the application to crash.
SO-02	Keep the application within its intended local-game behaviour and avoid unintended access to unrelated files or execution of operating-system commands.

5.1.2 Security Requirements

ID	Security requirement	Verification / acceptance criteria	Test
SR-01	The system shall ignore unsupported keyboard keys without crashing.	Unsupported keys are ignored and gameplay remains responsive.	TC-13
SR-02	The system shall not execute operating-system commands in response to player input.	Code review finds no player-input path that executes operating-system commands.	TC-14
SR-03	The system shall not read or modify unrelated user files during normal gameplay.	Code review finds no unrelated user-file access during normal gameplay.	TC-14
SR-04	The system shall validate direction changes against the current game state.	Immediate reverse-direction input is rejected.	TC-13
SR-05	The system shall handle window-close events and shutdown safely.	Closing the window exits without a crash.	TC-07

6. Quality Attributes and Acceptance Tests

6.1 Quality Attributes

The quality requirements cover performance (NFR-01), usability (NFR-02), reliability (NFR-03), maintainability (NFR-04), compatibility (NFR-05), documentation (NFR-06) and safe input/shutdown behaviour (SR-01 to SR-05).

6.2 Acceptance Criteria / Exit Criteria

Acceptance is determined by executing the corresponding tests in the separate Test Plan. The team should verify the requirements, resolve critical defects and record actual results. Requirements must not be marked as passed before the relevant test or review has been completed.

6.3 Acceptance Test Summary

Test area	What is verified	Related requirements	Current status
Startup and display	Start action, play area and initial snake state	FR-01 to FR-03	Pass (reported) — TC-01
Input and movement	Direction keys, movement and reverse-direction prevention	FR-04 to FR-06; SR-01; SR-04	Pass (reported) — TC-02 and TC-13
Food and score	Food placement, collection, growth, scoring and replacement food	FR-07 to FR-11	Pass (reported) — TC-03 and TC-04
Collision and game over	Boundary/self-collision, stopping and final score	FR-12 to FR-15	Pass (reported) — TC-05
Restart and shutdown	Restart, reset and safe close	FR-16 to FR-18; SR-05	Manual pass reported — TC-06 and TC-07; sanitizer step unconfirmed
Quality attributes	Performance, usability, reliability, maintainability, compatibility and documentation	NFR-01 to NFR-06	Pass (reported) — TC-08 to TC-12
Security review	No input-driven command execution or unrelated file access	SO-01 to SO-02; SR-01 to SR-05	Partial — TC-14; runtime Process Monitor trace and full review pending

Automated verification note

The captured CTest result reports 68 tests passed, 0 failed (100% pass), with total real test time of 25.14 seconds. The test run was executed in PowerShell after adding the SFML 2.6.2 runtime directory to PATH. This automated result does not replace separate manual and security procedures in the Test Plan.

7. UML Use-Case Diagram

The diagram shows the Player actor and the principal interactions with the Snake Game system.

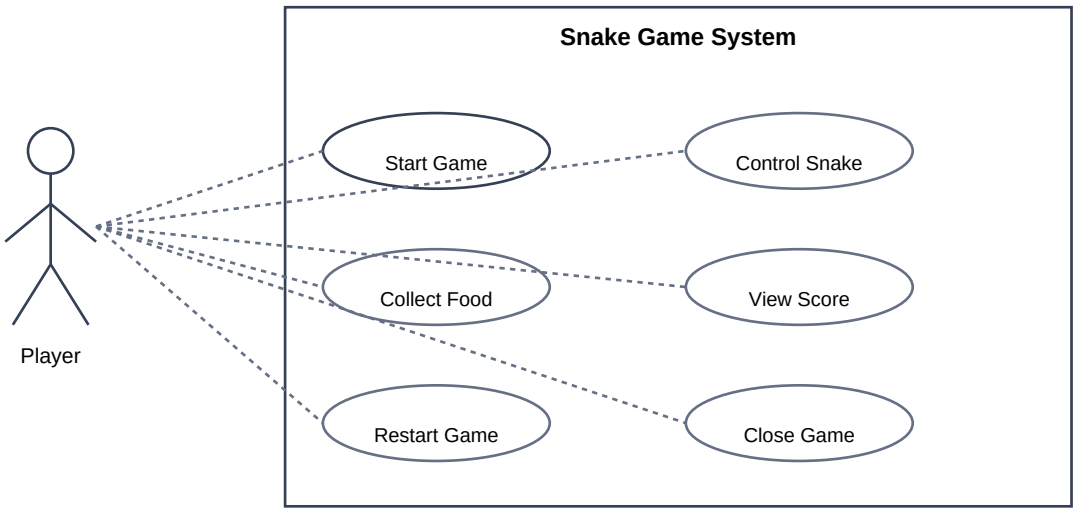

Figure 1. Player interactions with the Snake Game system.

7.1 Diagram Description

The Player starts a game, controls the snake, collects food through gameplay, views the score, restarts after game over and closes the game. The system boundary is labelled Snake Game System; the actor is outside the boundary and the use cases are inside it.

8. Requirements Traceability Matrix (RTM)

Status key: N = Not Run; P = Pass (reported); IP = In Progress / Partial; F = Fail; NA = Not Applicable. Reported statuses reflect the available execution update and are not a substitute for instructor or team approval. TC-14 remains partial; SR-02 and SR-03 must not be marked Pass until the full review and runtime Process Monitor evidence are completed.

Requirement ID	Requirement summary	SRS section / module	Test reference	Status
FR-01	Start new round	Section 4 / game control	TC-01	P
FR-02	Display bounded play area	Section 4 / rendering	TC-01	P
FR-03	Initial snake placement	Section 4 / snake setup	TC-01	P
FR-04	Keyboard direction input	Section 4 / input control	TC-02	P
FR-05	Snake movement	Section 4 / movement	TC-02	P
FR-06	Reject immediate reversal	Sections 4, 5.1 / input control	TC-02, TC-13	P
FR-07	Valid food placement	Section 4 / food	TC-03	P
FR-08	Detect food collection	Section 4 / food	TC-03	P
FR-09	Grow snake after food	Section 4 / snake	TC-03	P
FR-10	Update score	Section 4 / score	TC-04	P
FR-11	Generate replacement food	Section 4 / food	TC-04	P
FR-12	Boundary collision	Section 4 / collision	TC-05	P
FR-13	Self-collision	Section 4 / collision	TC-05	P
FR-14	Stop after fatal collision	Section 4 / game state	TC-05	P
FR-15	Show game over and final score	Section 4 / user interface	TC-05	P
FR-16	Restart after game over	Section 4 / game control	TC-06	P
FR-17	Reset state for new round	Section 4 / game control	TC-06	P

Requirement ID	Requirement summary	SRS section / module	Test reference	Status
FR-18	Safe application close	Sections 3, 4 / window events	TC-07	P
NFR-01	Performance at 30 FPS target	Section 5 / rendering loop	TC-08	P
NFR-02	Controls and restart documented	Section 5 / UI and README	TC-09	P
NFR-03	10 restart cycles without crash	Section 5 / game lifecycle	TC-10	P
NFR-04	Responsibilities identifiable in code	Section 5 / code structure	TC-11	P
NFR-05	Clean build and launch	Section 5 / build environment	TC-12	P
NFR-06	README has build/run instructions	Section 5 / documentation	TC-12	P
SR-01	Ignore unsupported keys safely	Section 5.1 / input handling	TC-13	P*
SR-02	No input-driven OS command execution	Section 5.1 / code review	TC-14	IP
SR-03	No unrelated user-file access	Section 5.1 / code review	TC-14	IP
SR-04	Validate direction changes	Section 5.1 / input handling	TC-13	P*
SR-05	Safe window-close and shutdown	Section 5.1 / lifecycle	TC-07	P*

* P* = Manual verification reported as passed; sanitizer sub-step remains unconfirmed.

9. Verification Limitations and Outstanding Actions

The SRS records reported test outcomes. The following activities remain open before final acceptance:

1. Complete TC-14 code review of every file-open/read/write call and capture a Windows Process Monitor trace through a full gameplay round, restart and application close.
2. Run or formally record the sanitizer-enabled verification required by TC-07 and TC-13.
3. Synchronize SRS Section 8, Test Plan Section 10 and RTM_SnakeGame.xlsx. Keep SR-02 and SR-03 at IP until TC-14 is complete.
4. Obtain actual team and instructor approvals; do not fill signatures or approval dates without approval.
5. Retain the automated test screenshot at reports/Automated_Test_Results_68_Tests.png and attach additional manual/security evidence to the repository.

10. Conclusion

The Snake Game automated test suite completed with 68 of 68 tests passing in the recorded CTest run. Reported manual checks indicate successful functional, performance, usability, reliability, maintainability and compatibility verification. Security acceptance is not complete: TC-14 remains partial, and sanitizer execution for TC-07 and TC-13 has not been confirmed. This SRS Version 1.1 is an implementation-aligned draft and remains subject to team and instructor approval.

Appendix A: Glossary

Term	Definition
Game-over state	The state entered after a fatal collision, in which the snake stops moving and the final score is shown.
Acceptance criterion	An observable condition used to decide whether a requirement is satisfied.
Traceability	A documented relationship between a requirement and its corresponding verification test.
Input latch	The rule allowing at most one accepted direction change between movement ticks.
RoundComplete	The stopped state entered when no free cell remains for replacement food.

Appendix B: References and Tools

1. Software Engineering Mini-Project Deliverables Part-1 instructions provided for the course.
2. SFML documentation for the version selected by the project team.
3. C++ compiler and language documentation relevant to the selected build environment.
4. Verification tools: Google Test 1.17.0, CTest, Git and GitHub. The repository README documents GitHub Actions CI; its latest run status must be checked on GitHub before submission.

End of SRS — Version 1.1 (Updated implementation-aligned draft)